

EFX₀ Allocations with at Most Three Relevant Goods: Existence and a Verified Polynomial-Time Algorithm

Evan Lin

University of Illinois Urbana-Champaign

Abstract. An allocation of indivisible goods is EFX₀ if no agent envies another agent’s bundle after removing any single good from it, even one the envious agent values at zero. We prove that every instance with nonnegative additive valuations in which each agent positively values at most three goods has a complete EFX₀ allocation, via a polynomial-time algorithm, K3ALG (for three relevant goods): a serial dictatorship whose leftover goods fill free slots, with the surplus going to one owner after at most one rotation of picks. For natural-number values, its correctness and a polynomial bound on its number of operations in an explicit unit-cost model are machine-checked in Lean 4 on the program itself; a bound on bit operations is written, not formalized. The existence results are machine-checked for values in any linearly ordered cancellative commutative monoid; instantiating them at $\mathbb{R}_{\geq 0}$ uses the standard fact that $\mathbb{R}_{\geq 0}$ is such a monoid (core Lean has no real numbers). If every agent values exactly three goods, none more than the other two together, there is a complete EFX₀ allocation in which all but at most one agent receive at most two goods.

Keywords: Fair division · Indivisible goods · EFX · Serial dictatorship · Formal verification · Lean

1 Introduction

Envy-freeness up to any good is a central relaxation of envy-freeness for indivisible goods [4]. Agents $N = [n]$ share goods $M = [m]$ with non-negative additive valuations $v_i(S) = \sum_{g \in S} v_i(g)$. An allocation X assigns every good to exactly one agent; agent i receives the bundle X_i , and no good is left out (allocations are *complete*). It is *EFX₀* if

$$v_i(X_i) \geq v_i(X_j \setminus \{h\}) \quad \text{for all } i \neq j \text{ and all } h \in X_j$$

(vacuous if $X_j = \emptyset$); *EFX* asks this only for h with $v_i(h) > 0$. The two differ only in which goods may be removed, so EFX₀ implies EFX, and the two coincide when all values are positive. *Under EFX₀, if X_j*

contains a good that i values at 0, then i must not envy X_j at all. When agents value few goods this binds: a good nobody values cannot go to an arbitrary agent, and one valued by j but not by i cannot join a bundle i envies.

For additive valuations, complete EFX allocations are known to exist only in special cases, among them (Table 2, Appendix H) three agents [5] and, more generally, three types of agents [7], $m \leq n + 3$ [13], four agents and at most nine goods [3], goods valued by at most two agents [1] (special cases in [6,14,8]), and hypergraphs of girth at least four [9], all for EFX_0 . For monotone valuations EFX can fail [2,12]. We limit instead how many goods an agent values: an instance is *k-relevant* if every agent positively values at most k goods. For $k \leq 2$ (a case of Theorem 1) serial dictatorship suffices, as formalized in [11]. For $k \leq 4$, Viswanathan and Mehta [15] announce ordinary EFX, with a proof sketch; their algorithm gives goods nobody values to an arbitrary agent, which can break EFX_0 . Such goods are easy to place (Lemma 5).

A 3-relevant instance with no complete EFX_0 allocation would refute the additive EFX conjecture (perturb all values by a small $\varepsilon > 0$; compare the perturbation lemma of [5]), but perturbing destroys the relevance structure, and no result we know of implies a positive answer [10]. We prove one, with a polynomial-time algorithm whose correctness and operation bound, in an explicit unit-cost model, are machine-checked; as EFX_0 implies EFX, this also proves the case $k = 3$ of [15] in full.

Theorem 1 (existence). *Every instance with $n \geq 1$ agents and non-negative real additive valuations in which every agent positively values at most three goods has a complete EFX_0 allocation.*

Theorem 2 (one large bundle). *For $n \geq 1$ agents with nonnegative real additive valuations, each valuing exactly three goods, none more than the other two together (balanced), there is a complete EFX_0 allocation in which all but at most one agent receive at most two goods.*

In the *unit-cost model*, each list step, comparison of agents or goods, read of an input value or table entry, and comparison or addition of natural numbers of any size costs one operation.

Theorem 3 (algorithm). *Given $n \geq 1$ agents, m goods and natural-number values in binary, $K3\text{ALG}$ (Section 3) performs at most $400(n + m + 1)^4$ operations in the unit-cost model, and if every agent positively values at most three goods, it returns a complete EFX_0 allocation.*

Table 1. The claims and their evidence (Section 5); steps are unit-cost operations. *For any linearly ordered cancellative commutative monoid; that $\mathbb{R}_{\geq 0}$ is one is a standard fact, not a Lean theorem. *Rankings*: only the rankings matter.

Claim	Values	Evidence
Thm. 1: existence	$\mathbb{R}_{\geq 0}$	Lean-checked theorem*
Thm. 2: one large bundle	$\mathbb{R}_{\geq 0}$	Lean-checked theorem*
Thm. 3: K3ALG is correct	$\mathbb{N}$	Lean-checked theorem
Thm. 3: $400(n + m + 1)^4$ steps	$\mathbb{N}$	Lean-checked theorem
Sect. 5: $270(n^4 + n^2m)$ steps	$\mathbb{N}$	Lean-checked theorem
Prop. 3: size of the large bundle	rankings	Lean-checked theorem
Cor. 1: rational values	$\mathbb{Q}_{\geq 0}$	Lean-checked theorem
Cor. 3: real values, oracle	$\mathbb{R}_{\geq 0}$	Lean-checked theorem*
Cor. 2: bits; Python: $O((n + m) \log n)$	$\mathbb{N}$	written, not formalized
App. F: existence for $n \leq 7$	$\mathbb{R}_{\geq 0}$	verified by exhaustive computation
App. D: speed of the Python version	$\mathbb{N}$	measurement

Corollary 1 (rational values). *If every agent positively values at most three goods, with nonnegative rational values, K3ALG run on each agent's values scaled to natural numbers (which preserves every comparison of sums of that agent's values) returns a complete EFX₀ allocation.*

Table 1 lists the claims, each with one kind of evidence. Appendix A reduces the problem to *cores*, the setting of the certification and the counterexamples; Appendix C has a state dictionary and traces two runs.

2 Preliminaries

Let $R_i = \{g : v_i(g) > 0\}$, the *relevant* goods of i . Agent i *strongly envies* X_j if $v_i(X_i) < v_i(X_j \setminus \{h\})$ for some $h \in X_j$, and is *safe* if it strongly envies no bundle; a bundle of at most one good is never strongly envied, and a good is *alone* if it is its bundle's only good. If $|R_i| = 3$ we write $R_i = \{a_i, b_i, c_i\}$ with $v_i(a_i) \geq v_i(b_i) \geq v_i(c_i) > 0$, fix this order (ties broken by index) as $a_i \succ_i b_i \succ_i c_i$, and call i *balanced* if $v_i(a_i) \leq v_i(b_i) + v_i(c_i)$, and *strictly balanced* if the inequality is strict. A valuation is *consistent* with these rankings if $v_i(a_i) \geq v_i(b_i) \geq v_i(c_i) > 0$ and $v_i(g) = 0$ for every other good.

Lemma 1 (peeling). *Let $i \in N$ and $P \subseteq M$ satisfy (i) $v_i(P) \geq v_i(M \setminus P)$ and (ii) $|P| \leq 1$ or P is worthless to every other agent. If the instance without i and P has an EFX₀ allocation Y , then Y with $X_i = P$ is EFX₀.*

Rule R1 applies Lemma 1 with $P = \{p\}$ for a favourite remaining good p of i when $v_i(p) \geq v_i(M \setminus \{p\})$, and with $P = \emptyset$ when $v_i(p) = 0$: then i

values every remaining good at zero, envies no bundle, and gets nothing, which constrains no one. If R1 fails for every agent, then $2v_i(g) \leq 2v_i(p) < v_i(M)$ for all g , so in a 3-relevant instance every agent values exactly three goods and is strictly balanced.

Lemma 2 (real values reduce to naturals; Appendix A). *If v is nonnegative real additive with $|R_i| \leq 3$ for all i , there are natural-number additive valuations w with the same relevant goods such that $v_i(S) \leq v_i(T) \iff w_i(S) \leq w_i(T)$ for every agent i and all $S, T \subseteq M$.*

3 Algorithm K3ALG

K3ALG has two stages: *Stage R* peels agents by rule R1, and *Stage L* runs construction LB^+ on the remaining agents A and goods M ; each agent of A then values exactly three goods and is strictly balanced.

Intuition. Stage L starts with a serial dictatorship, Phase 1, that mimics R1: an agent with at most two goods left takes the better one; only if every waiting agent has all three does one, a *leader*, take its top. Without leftover goods every bundle would be one pick, which nobody strongly envies. The difficulty is the leftovers: an agent that holds one good and lost better ones to earlier agents is safe only if those sit alone. For strictly balanced i with distinct values, Lemma 7 (Appendix G) gives the exact cases: holding only b_i , i is safe iff a_i is alone; only c_i , iff a_i and b_i are; $\{b_i, c_i\}$, always; a_i , unless b_i, c_i share a bundle of three or more goods. *Needs* record the goods that must sit alone; a *frozen* agent holds one and takes nothing else; an *upgrade* moves an agent from b_i to $\{b_i, c_i\}$, releasing a_i and using up a leftover; the *owner* takes the junk that does not fit the slots, and its bundle, the only large one, must respect the last case. The first candidate owner is the last agent r not upgraded: nobody after r needs its pick, and each agent that r 's bundle could threaten leads a block, which supplies a slot for a protecting good unless it is r 's own; if that fails, one rotation along a chain from that block's leader k^* to r makes k^* the owner.

Definitions (dictionary and examples: Appendix C). Let G be the set of goods not yet picked (initially M). At its turn, agent i takes its $\succ_i$ -favourite good of $R_i \cap G$ as its *pick* Y_i , or no pick ($Y_i = \perp$). The next agent to move has at most two of its goods in G if some unprocessed agent does (an *R1 step*); otherwise it finds all three available (an *insertion step*); this is *R1 priority*. A *block* is an insertion step with the R1 steps after it; its

inserted agent is its *leader*. Step L3 then also gives c_u to some agents u whose pick is b_u ; these *upgraded* agents form U . The *junk* J is the set of goods still unallocated after Phase 1 and the upgrades. For $i \notin U$ the *needs* are $N_i = \{g \in R_i : g \succ_i Y_i\}$ ($N_i = R_i$ if $Y_i = \perp$), and $NA = \bigcup_{i \notin U} N_i$ is the set of goods *needed alone*. An agent $i \notin U$ is *frozen* if $Y_i \in NA$, and a *terminal* otherwise. The *base* of i is $\{Y_i\}$ (or $\emptyset$) for $i \notin U$ and $\{b_i, c_i\}$ for $i \in U$. A terminal may add junk up to two goods: its *slots* are $\text{cap}(i) = 1$ or 2 for a terminal with or without a pick, and $\text{cap}(i) = 0$ for other agents; $S = \sum_i \text{cap}(i)$. A *completion* gives every agent its base and each terminal i at most $\text{cap}(i)$ junk goods; *without owner*, this places all junk, which needs $|J| \leq S$; *with owner* o (a terminal or upgraded agent), o also takes the rest of the junk. It satisfies the *owner constraint* (OC) if no agent $x \notin U$, $x \neq o$, with $Y_x = a_x$ has $\{b_x, c_x\} \subseteq X_o$. The agents *exposed* for o are $E_o = \{x \notin U, x \neq o : Y_x = a_x, \{b_x, c_x\} \subseteq J \cup \text{base}(o)\}$; $\pi_x = \{b_x, c_x\} \cap J$. A *need chain* from a frozen x is a sequence $x = x_0, \dots, x_t$ of distinct agents in which each $x_{s+1} \notin U$ needs Y_{x_s} , $x_0, \dots, x_{t-1}$ are frozen, and x_t is a terminal.

Algorithm K3ALG (as formalized in EFX.K3.algoC). Input: $n \geq 1$ agents, m goods, a table of values $v_i(g) \in \mathbb{N}$, each entry read at unit cost; A and M start as all agents and goods. Lists are in index order; “first” means smallest index.

R. While $|A| \geq 2$ and $M \neq \emptyset$: find the first $i \in A$ such that, for the first good $p \in M$ of largest $v_i(p)$, $v_i(p) = 0$ or $v_i(M \setminus \{p\}) \leq v_i(p)$; if none, go to L1; else remove i from A , and if $v_i(p) > 0$, let $X_i = \{p\}$ and $M \leftarrow M \setminus \{p\}$. Then give M to the agent of A , if $|A| = 1$, and stop.

L1. For $i \in A$, sort i ’s relevant goods in M as a_i, b_i, c_i (ties by index).

L2. Phase 1. Repeatedly, the first unprocessed agent with at most two of its goods unpicked (else the first unprocessed agent) picks the first of a_i, b_i, c_i still unpicked, if any.

L3. Upgrades. $U \leftarrow \emptyset$. While some $k \notin U$ has $Y_k = b_k, c_k \in J$ and $b_k \notin NA$ (for the current U), add the first such k to U .

L4. If $|J| \leq S$, return COMPLETE(no owner, $\emptyset$).

L5. Let r be the last-processed agent not in U (Lemma 4), and $H = \text{HitSet}(E_r)$. If $|H| \leq S - \text{cap}(r)$, return COMPLETE(r, H).

L6. Rotation. Let k be the first agent of E_r in r ’s block. Scanning the agents processed after k in order, append j to the chain $k, \dots$ if the chain’s last agent is frozen, $j \notin U$, and j needs the pick of the chain’s last agent. Each chain agent takes its predecessor’s pick; k takes the pick b_k and the upgrade good c_k and joins U .

L7. Recompute J , cap and S . If $|J| \leq S$, return $\text{COMPLETE}(\text{no owner}, \emptyset)$; otherwise return $\text{COMPLETE}(k, \text{HITSET}(E_k))$.

$\text{HITSET}(E)$: if two agents $x \neq y$ of E share a good of $\pi_x \cap \pi_y$ (the first such x , then y , then the first such good among b_x, c_x), return that good followed by, for each other $z \in E$ in order, b_z if $b_z \in J$ and c_z otherwise; else return that list for all $z \in E$. As each $\pi_z \neq \emptyset$ (proofs of Theorems 5 and 6), all its entries are junk, and it has at most $|E| - 1$ entries if two sets meet, else $|E|$ ($\text{EFX.LB.hitSet_length}$). $\text{COMPLETE}(o, H)$: picks go to their pickers, c_u to each $u \in U$; the junk, H first and then the rest of J , fills the slots of the agents other than o in index order, each good at its first occurrence (so a repeat in H is harmless); o gets the remaining junk.

4 Correctness

Stage R is correct by Lemma 1, for any agent R1 applies to. For Stage L we prove more, for every Phase 1 order with R1 priority, upgrade order and need chain (construction LB^+), so K3ALG's smallest-index tie-breaking is not needed for correctness. Invariants of Phase 1 (Appendix B): (I1) if $g \in R_i$ and $g \succ_i Y_i$ (or $Y_i = \perp$), then g was picked before i 's turn, and (B) by an agent of i 's block; (I2) if $g \in R_i$ is never picked, then $Y_i \succ_i g$; (I3) an agent that finds all its goods available is a leader. R1 priority gives (B) and (I3), used in Theorem 5; the upgrades' stopping condition (UT) is used in Theorem 6. A *pre-allocation* is a state (Y, U) of picks and upgraded agents (Appendix B); it is *valid* if (V1) $J \cap NA = \emptyset$ and (V2) $b_u, c_u \notin NA$ for $u \in U$; after L3 it is (Lemma 6).

Theorem 4 (soundness). *Let (Y, U) be valid. Every completion satisfying (OC) is EFX_0 for every balanced additive valuation consistent with the rankings; only the owner's bundle can exceed two goods.*

Proof (sketch). A non-owner holds its base and at most $\text{cap}(i)$ junk goods, at most two goods in all (a frozen one only its pick). So a bundle of two or more goods consists of a terminal's pick (not in NA by definition), goods b_u, c_u (not in NA by (V2)) and junk (not in NA by (V1)): it has no good needed alone. Fix i and such a bundle X_j . If $i \in U$, X_j meets R_i at most in a_i , and $v_i(a_i) \leq v_i(b_i) + v_i(c_i)$ by balance. If $i \notin U$, the goods i prefers to its pick are in $N_i \subseteq NA$, so X_j meets R_i only below Y_i (not at all if $Y_i = \perp$). One such good is worth at most $v_i(Y_i)$; two exist only if $Y_i = a_i$ and $\{b_i, c_i\} \subseteq X_j$. If $|X_j| = 2$, removing one leaves at most $v_i(a_i)$; if $|X_j| \geq 3$, then $j = o$, and (OC) excludes exactly this (Appendix B). $\square$

Lemma 3 (owner criterion). *Let (Y, U) be valid with $|J| > S$, and let o be a terminal or an upgraded agent. If no $x \in E_o$ has $\{b_x, c_x\} \subseteq \text{base}(o)$, and some $H \subseteq J$ with $|H| \leq S - \text{cap}(o)$ meets $\{b_x, c_x\}$ for every $x \in E_o$, then filling the other slots with H first gives a completion with owner o that satisfies (OC); o is a valid owner.*

Lemma 4 (the candidate r). *Some agent is not upgraded, and the last-processed such agent r is a terminal: the first agent picks its top, so is not upgraded, and a $j \notin U$ needing Y_r would come after r by (I1) (Appendix B).*

Theorem 5 (the owner r). *Let $|J| > S$ after Phase 1 and the upgrades. Then r is a valid owner unless the bad case holds: an agent k^* of r 's block is exposed for r and frozen, every need chain from k^* ends at r , and the sets π_x ($x \in E_r$) are pairwise disjoint.*

Proof (sketch). Let $x \in E_r$. Each of b_x, c_x is junk, so never picked, or Y_r , picked after x ; so x found all its goods available and is a leader (I3). So exposed agents lead distinct blocks, at most one, k^* , in r 's block, and $\pi_x \neq \emptyset$. The count $S - \text{cap}(r) \geq |E_r| - 1$ comes from an injection $x \mapsto \tau(x)$ from the exposed agents outside r 's block into the terminals other than r : $\tau(x) = x$ if x is a terminal, and otherwise the end of a need chain from x , which exists and stays in x 's block, as by (B) an agent that needs a frozen agent's pick comes later in the same block. Distinct blocks give distinct terminals, each with $\text{cap} \geq 1$. One good of each π_x , one shared good if two sets meet, then fits in $S - \text{cap}(r)$ unless k^* exists, is frozen, has no need chain to another terminal, and the π_x are disjoint (Appendix B). $\square$

Theorem 6 (rotation). *In the bad case, rotate along any need chain $k^* = x_0, \dots, x_t = r$: each x_s ($s \geq 1$) takes $Y_{x_{s-1}}$ as its pick, and k^* joins U with pick b_{k^*} and upgrade good c_{k^*} . The result is valid, and if its junk exceeds its slots, then k^* is a valid owner.*

Proof (sketch). Chain agents get goods they needed, so NA shrinks, and $b_{k^*}, c_{k^*} \in J \cup \{Y_r\}$ are free. Agents exposed for k^* afterwards were exposed for r ((UT) excludes r), disjointness keeps their pairs out of $\{b_{k^*}, c_{k^*}\}$, and their terminals $\tau(x)$ lie outside r 's block and keep their slots (Appendix B; Example 3). $\square$

Theorem 7 (LB^+ never fails). *If every agent values exactly three goods and is balanced, LB^+ (Stage L with any Phase 1 order with R1 priority, upgrade order and need chain) returns a complete allocation, EFX₀ for every balanced additive valuation consistent with the rankings, with at most one bundle of more than two goods (by Lemmas 6, 3 and Theorems 4–6).*

Proof (of Theorems 1–3: correctness). If L5’s test fails, r is not a valid owner (Proposition 1), so the bad case holds (Theorem 5); L6’s k is k^* , its chain is a need chain from k^* , so ends at r , and Theorem 6 applies to L7. So Stage L runs LB^+ with R1 priority (Theorem 7, hence Theorem 2); with Lemma 1 and induction on n , K3ALG is correct; Lemma 2 extends existence to real values. $\square$

5 Running Time and Verification

Proposition 1 (the owner test). *With $|J| > S$ after Phase 1 and the upgrades, r is a valid owner if and only if $|E_r| \leq S - \text{cap}(r)$ or two of the sets π_x ($x \in E_r$) meet; $\text{HITSET}(E_r)$ is then a witness.*

Indeed, the sets π_x are nonempty with at most two goods each, such sets have a hitting set of fewer than $|E_r|$ goods iff two meet, and $S - \text{cap}(r) \geq |E_r| - 1$ (proof of Theorem 5). Running time has three layers (Appendix D): (i) Theorem 3’s Lean-checked bound for `EFX.K3.algoC`, a *cost monad* program (a value with a count) whose value is `EFX.K3.algo`; (ii) a Lean-checked finer count, $270(n^4 + n^2m)$, whose n^4 term comes from L3; (iii) an unverified Python implementation, with a written bound $O((n+m)\log n)$ after the input is read, and measured times. Corollary 2 (written, not formalized) bounds the bit operations.

Verification. The formalization uses core Lean 4 (v4.34.0), no packages and no `sorry`; every declaration depends only on the axioms `propext`, `Classical.choice` and `Quot.sound`. The trusted base is the cost model, six definitions from [11] and, for other value types, their counterparts over the class `EFX.OrderedValue` (Appendix E). Section 4 is formalized in full, the other results as Table 1 states. Exhaustive computation over all connected cores and all 6^n strict profiles (ties reduce to these, Appendix G) verifies Theorem 1 for $n \leq 7$ (Appendix F).

6 Conclusion

EFX_0 with four relevant goods per agent is open (for ordinary EFX, see [15]); there, one rotation of the kind used here no longer suffices, already with one agent valuing four goods ($n = 5$, [10]). The least size of the large bundle is open too.

Acknowledgments. The proofs, code and Lean formalization were developed with the assistance of AI coding agents (Claude Code).

References

1. Afshinmehr, M., Ashuri, A., Mahmoudkhan, P., Mehlhorn, K., Shahrezaei, A.M.: EFX allocations exist on multi-graphs. arXiv preprint arXiv:2606.18665 (2026), accepted at the 27th ACM Conference on Economics and Computation (EC 2026)
2. Akrami, H., Mayorov, A., Mehlhorn, K., Srinivas, S., Weidenbach, C.: A counterexample to EFX $n \geq 3$ agents, $m \geq n + 5$ items, submodular valuations via SAT-solving. arXiv preprint arXiv:2604.18216 (2026)
3. Alkassar, E., Fouz, M., Mehlhorn, K.: Complete EFX allocations exist for four additive agents and up to nine goods. arXiv preprint arXiv:2608.08590 (2026)
4. Amanatidis, G., Birmpas, G., Filos-Ratsikas, A., Voudouris, A.A.: Fair division of indivisible goods: A survey. In: Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence (IJCAI-22). pp. 5385–5393 (2022). <https://doi.org/10.24963/ijcai.2022/756>, arXiv:2202.07551
5. Chaudhury, B.R., Garg, J., Mehlhorn, K.: EFX exists for three agents. Journal of the ACM **71**(1), 4:1–4:27 (2024). <https://doi.org/10.1145/3616009>, conference version in Proc. 21st ACM Conference on Economics and Computation (EC 2020), pp. 1–19; full version arXiv:2002.05119
6. Christodoulou, G., Fiat, A., Koutsoupias, E., Sgouritsa, A.: Envy-freeness up to any good allocations on graphs. Mathematics of Operations Research (2026). <https://doi.org/10.1287/moor.2025.0915>, conference version: “Fair Allocation in Graphs”, in Proc. 24th ACM Conference on Economics and Computation (EC 2023), pp. 473–488
7. HV, V.P., Ghosal, P., Nimbhorkar, P., Varma, N.: EFX exists for three types of agents. In: Proceedings of the 26th ACM Conference on Economics and Computation (EC 2025). pp. 101–128. ACM (2025). <https://doi.org/10.1145/3736252.3742509>, arXiv:2410.13580
8. Kaviani, A., Seddighin, M., Shahrezaei, A.: Almost envy-free allocation of indivisible goods: A tale of two valuations. In: Web and Internet Economics (WINE 2024). pp. 261–276. Lecture Notes in Computer Science, Springer (2026). https://doi.org/10.1007/978-3-032-08560-3_15, full version arXiv:2407.05139
9. Lianas, T., Sgouritsa, A., Sotiriou, M.M.: EFX allocation in (multi)hypergraphs. Proceedings of the AAAI Conference on Artificial Intelligence **40**(20), 17102–17110 (2026). <https://doi.org/10.1609/aaai.v40i20.38759>, AAAI-26; arXiv:2608.03171
10. Lin, E.: efx0-three-goods: EFX₀ with at most three relevant goods per agent (proofs, certificates, Lean formalization). GitHub repository, commit `fa08f55`, <https://github.com/evanlin23/efx0-three-goods> (2026)
11. Lin, E., Osterhaus, G.: Strong EFX allocations for 2-relevant agents: Lean 4 formalization. GitHub repository `mrd-efx`, tag `v1.2.0` (commit `fa42aa1`, 23 September 2026), <https://github.com/evanlin23/mrd-efx> (2026)
12. Mackenzie, S., Suzuki, M.: Counterexamples to EFX for submodular and subadditive valuations. arXiv preprint arXiv:2605.06451 (2026)
13. Mahara, R.: Extension of additive valuations to general valuations on the existence of EFX. Mathematics of Operations Research **49**(2), 1263–1277 (2024). <https://doi.org/10.1287/moor.2022.0044>, conference version in Proc. 29th Annual European Symposium on Algorithms (ESA 2021), LIPIcs 204, 66:1–66:15; arXiv:2107.09901
14. Sgouritsa, A., Sotiriou, M.M.: On the existence of EFX allocations in multigraphs. arXiv preprint arXiv:2502.09777 (2025), extended abstract, without the bipartite

- case, in Proc. 24th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2025), pp. 2735–2737
15. Viswanathan, M., Mehta, R.: On the existence of EFX under picky or non-differentiative agents. In: Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2024). pp. 2534–2536 (2024). <https://doi.org/10.65109/osos1146>, extended abstract

A Proofs for Section 2 and Cores

Full proofs of all reduction lemmas are in `proofs/lemmas.md` of [10]; we reproduce those used here.

Proof (of Lemma 1). Every other bundle lies in $M \setminus P$, so i values it at most $v_i(M \setminus P) \leq v_i(P) = v_i(X_i)$. Towards P , every other agent sees at most one good or a bundle worthless to it (and nothing if $P = \emptyset$). The remaining constraints are those of Y . $\square$

Lemma 5 (unvalued goods). *Let $Z \neq \emptyset$ be the set of goods valued by no agent. If the instance without Z has an EFX₀ allocation, so does the instance.*

Proof. Let Y be EFX₀ for $M \setminus Z$. While the envy graph ($i \rightarrow j$ iff $v_i(Y_j) > v_i(Y_i)$) has a cycle, give each agent on it the bundle of its successor. The bundles are the same and no agent loses value; an agent that now sees its old bundle satisfies $v_i(Y_i \setminus \{h\}) \leq v_i(Y_i)$, so EFX₀ is preserved. The sum of values strictly increases, so this stops, with an acyclic envy graph. Let s be a source: $v_i(Y_s) \leq v_i(Y_i)$ for all i . Put $X_s = Y_s \cup Z$. For $i \neq s$, $v_i(X_s \setminus \{h\}) \leq v_i(X_s) = v_i(Y_s) \leq v_i(Y_i)$; all other constraints are unchanged. $\square$

Cores. Agent i is *top-heavy* if $v_i(a_i) \geq v_i(b_i) + v_i(c_i)$. Rule *R2* gives i the set P of its relevant goods that no other agent values, if $|P| \geq 2$ and (i) of Lemma 1 holds. A *core* is an instance with at least two agents, each valuing exactly three goods and strictly balanced, each with at most one *private* good (one no other agent values), and no good valued by nobody. A core is *connected* if its agents cannot be split into two nonempty groups that value disjoint sets of goods. Cores are the instances that the reductions below cannot simplify; the exhaustive certification (Appendix F) and the counterexamples (Appendix G) are about cores.

Proposition 2 (core reduction). *If every core with at most N_0 agents has an EFX₀ allocation, so does every 3-relevant instance with between 1 and N_0 agents.*

Proof (of Proposition 2). Induction on $n + m$ over 3-relevant instances with $1 \leq n \leq N_0$. One agent takes everything. Otherwise: (1) if some good is valued by nobody, use Lemma 5; (2) else if some agent has at most two relevant goods or is top-heavy, use rule R1 and Lemma 1; (3) else if some agent i has at least two private goods, let P be all of them:

if $|P| = 3$, (i) is trivial, and if $P = \{x, y\}$ and $R_i \setminus P = \{z\}$, then $v_i(x) + v_i(y) \geq v_i(b_i) + v_i(c_i) > v_i(a_i) \geq v_i(z)$, so Lemma 1 applies (rule R2); (4) else the instance is a core, covered by hypothesis. Each case passes to a smaller 3-relevant instance. $\square$

In Lean this is `EFX.core_reduction` (with `EFX.peel`, `EFX.peelR2` and `EFX.junk`).

Proof (of Lemma 2). Fix i ; w_i is built from v_i alone. Goods outside R_i contribute 0 to both sides under both valuations, and removing goods common to S and T changes neither comparison, so it suffices that w_i preserves $v_i(X) \leq v_i(Y)$ for disjoint $X, Y \subseteq R_i$. Name the relevant goods with values $a \geq b \geq c > 0$. Comparisons with an empty side are decided by positivity; two single goods by the ties among a, b, c ; a single good against the other two by the sign of $a - (b + c)$, because $b < a + c$ and $c < a + b$ always hold. Two disjoint pairs do not fit in three goods. Choose w_i on R_i (in the same order) by the pattern: (4, 3, 2) if $a > b > c$ and $a < b + c$; (3, 2, 1) if $a > b > c$ and $a = b + c$; (5, 2, 1) if $a > b > c$ and $a > b + c$; (2, 2, 1) if $a = b > c$; (3, 2, 2), (2, 1, 1), (3, 1, 1) if $a > b = c$ and $a < 2b$, $a = 2b$, $a > 2b$; (1, 1, 1) if $a = b = c$; (2, 1) or (1, 1) for two relevant goods; (1) for one; and $w_i(g) = 0$ for $g \notin R_i$. Each triple has the same ties and the same sign of $a - (b + c)$ as its pattern. $\square$

Lean: `EFX.112`, for values in any linearly ordered cancellative commutative monoid, which uses the 31 reorderings of these patterns (the table `EFX.Pat.reps`, checked by `decide`) instead of sorting, and `EFX.efx0_iff_of_agree` for the transfer of `EFX0`. With a comparison oracle, six comparisons of subset sums, each chosen after the previous answers, suffice to choose the pattern (sorting a, b, c first and then settling the ties and the sign of $a - (b + c)$ can take seven); the formalized program of Corollary 3 asks all twelve instead.

B Proofs for Section 4

These are the proofs of `proofs/lb_last_step.md` of [10], Sections 1–6. A *pre-allocation* is a pair $P = (Y, U)$ in which each Y_i is a good of R_i or $\perp$, no good is picked twice, and each $u \in U$ has $Y_u = b_u$ and an upgrade good c_u that is nobody’s pick and no other upgraded agent’s; J is the set of the other goods. Phase 1 with the upgrades yields one, and so does the rotation (step (a) below). F and T denote its frozen agents and its terminals, and $\omega = |J| - S$. After the rotation, primes denote the new state: $P' = (Y', U')$, J' , NA' , cap' , S' , ω' , E'_k .

Phase 1. (I1): i took its favourite remaining good, and goods leave G only by being picked. (I2): g was still in G at i 's turn. (I3): R1 steps process only agents with at most two goods left. (B): each block starts with an insertion step, when every unprocessed agent x has $R_x \subseteq G$ (by R1 priority); by (I1) g was picked before i 's turn, and not in an earlier block, since i was unprocessed when its own block started.

Proof (of Theorem 4). Only X_o can exceed two goods. Fix i and $j \neq i$ with $|X_j| \geq 2$. Then j is not frozen, so $X_j \cap NA = \emptyset$ by (V1) and (V2). If $i \in U$, then X_j meets R_i in at most a_i , and $v_i(X_j \setminus \{h\}) \leq v_i(a_i) \leq v_i(b_i) + v_i(c_i) \leq v_i(X_i)$. If $Y_i = \perp$, then $R_i = N_i \subseteq NA$ and X_j is worthless to i . Otherwise every good of $R_i \cap X_j$ is outside $N_i \cup \{Y_i\}$, hence below Y_i . If there is at most one, $v_i(X_j \setminus \{h\}) \leq v_i(Y_i) \leq v_i(X_i)$. If there are two, then $Y_i = a_i$ and $R_i \cap X_j = \{b_i, c_i\}$; for $|X_j| = 2$ removing one leaves a good worth at most $v_i(a_i)$, and $|X_j| \geq 3$ means $j = o$, excluded by (OC). $\square$

Proof (of Lemma 3). Since $|J| = S + \omega > S - \text{cap}(o)$, H extends to $C \subseteq J$ with $|C| = S - \text{cap}(o)$, which exactly fills the slots of the terminals other than o ; let $X_o = \text{base}(o) \cup (J \setminus C)$. If $x \notin U$, $x \neq o$, $Y_x = a_x$ had $\{b_x, c_x\} \subseteq X_o$, then $\{b_x, c_x\} \subseteq J \cup \text{base}(o)$, so $x \in E_o$, and $H \subseteq C$ meets $\{b_x, c_x\}$ outside X_o , a contradiction. $\square$

Lemma 6 (upgrades). *When the upgrade loop stops, (Y, U) is a valid pre-allocation, and (UT) no $k \notin U$ has $Y_k = b_k$, $c_k \in J$ and $b_k \notin NA$.*

Proof. (UT) is the stopping condition. (V1): by (I2) a junk good lies below the pick of every agent that values it, so it is in no N_i . (V2): c_u was junk; $b_u \notin NA$ held when u was upgraded, and NA only shrinks as U grows. $\square$

Proof (of Lemma 4 and Theorem 5). (A0) r exists: the first processed agent is inserted and holds its top, so it is not upgraded; in Lean, `EFX.LB.lastOut_exists`. (A1) $r \in T$: if $Y_r = \perp$, r is not frozen by definition; if $Y_r \in N_j$ for some $j \notin U$, then by (I1) j was processed after r , contradicting the choice of r (`EFX.LB.lastOut_terminal`). (A2) Every agent processed after r is in U , by the choice of r . (A3) Let $x \in E_r$. Then x was processed before r (A2). At x 's turn, $a_x = Y_x$, every junk good, and Y_r were available, and $\{b_x, c_x\} \subseteq J \cup \{Y_r\}$, so $R_x \subseteq G$ and x is a leader by (I3). Hence each block contains at most one exposed agent; let B^* be r 's block and k^* its exposed agent, if any. (A4) For a frozen x , some $j \notin U$ has $Y_x \in N_j$, and by (B) j is processed after x in x 's block.

Iterating while the current agent is frozen gives a need chain of distinct agents of x 's block ending at a terminal. Let $\tau(x) = x$ if $x \in T$, and otherwise the end of some need chain from x ; it has $\text{cap} \geq 1$. For $x \in E_r$, $\pi_x = \{b_x, c_x\} \cap J \neq \emptyset$, since at most one of b_x, c_x is Y_r ; for the same reason no exposed pair lies in $\text{base}(r)$, the first condition of Lemma 3.

The agents $\tau(x)$, $x \in E_r \setminus B^*$, lie in distinct blocks other than B^* , so they are distinct terminals other than r , and $S - \text{cap}(r) \geq |E_r \setminus B^*|$. Let H contain one good of each π_x , a common one when two meet. If $E_r \cap B^* = \emptyset$, then $|H| \leq |E_r| \leq S - \text{cap}(r)$. Otherwise $E_r \cap B^* = \{k^*\}$, $k^* \neq r$ and $|E_r \setminus B^*| = |E_r| - 1$. If $k^* \in T$, or some need chain from k^* ends at a terminal other than r , then $\tau(k^*)$ can be chosen in $B^* \setminus \{r\}$, one more terminal, so $S - \text{cap}(r) \geq |E_r| \geq |H|$. If two sets π_x meet, then $|H| \leq |E_r| - 1 \leq S - \text{cap}(r)$. If none of these holds, the bad case holds. $\square$

Proof (of Theorem 6). Let $k = k^*$, $W = J \cup \{Y_r\}$; the new junk is $J' = W \setminus \{b_k, c_k\}$, and every pick other than Y_r is still a pick in P' . In P' , $k \in U'$ with $Y'_k = b_k$ and upgrade good c_k , so $Y'_u = b_u$ holds for every $u \in U'$. (a) P' is a pre-allocation: $Y'_{x_s} \in R_{x_s}$ since x_s needed it, and $b_k, c_k \in W$ are neither picks of P' other than Y'_k nor goods c_u with $u \in U$. (b) $NA' \subseteq NA$: $Y'_{x_s} \succ Y_{x_s}$ (or $Y_{x_s} = \perp$), so $N'_{x_s} \subseteq N_{x_s}$; k had $N_k = \emptyset$ and is now in U' . (c) Validity: $J' \subseteq W$, $J \cap NA = \emptyset$ and $Y_r \notin NA$ (A1), so $J' \cap NA' = \emptyset$; for $u \in U$, $b_u, c_u \notin NA \supseteq NA'$; $b_k, c_k \in W$ are not in NA . (d) A terminal $x \notin B^*$ keeps its pick and is a terminal of P' with the same slots. (e) $E'_k \subseteq E_r \setminus \{k\}$, where E'_k is taken in P' , with $J' \cup \{b_k, c_k\} \subseteq W$. An agent off the chain in E'_k is in E_r . An interior chain agent x_s ($1 \leq s < t$) is not: $a_{x_s} = Y'_{x_s} \succ Y_{x_s}$, so $Y_{x_s} \in \{b_{x_s}, c_{x_s}\}$, and $Y_{x_s} \notin W$. Nor is r : $Y_r \neq a_r$, since r needs $Y_{x_{t-1}}$. If $Y_r = \perp$ or $Y_r = c_r$, then b_r was picked by another agent (I1), so $b_r \notin W$. If $Y_r = b_r$, then (UT) applies to r ($b_r \notin NA$ by (A1)), so $c_r \notin J$, and $c_r \neq Y_r$, so $c_r \notin W$. (f) If some $x \in E'_k$ had $\{b_x, c_x\} = \{b_k, c_k\}$, then $x \in E_r \setminus \{k\}$ and $\pi_x = \pi_k \neq \emptyset$, against disjointness. So every $x \in E'_k$ has a good of $\{b_x, c_x\}$ in J' : $\text{HitSet}(E'_k)$ in L7 lists junk goods. (g) Let H' contain one good of $\{b_x, c_x\} \cap J'$ for each $x \in E'_k$. By (e), $|H'| \leq |E_r| - 1 = |E_r \setminus B^*|$. The terminals $\tau(x)$, $x \in E_r \setminus B^*$, are distinct, outside B^* , and keep $\text{cap}' \geq 1$ by (d). So $|H'| \leq S' = S' - \text{cap}'(k)$, and Lemma 3 applies to P' and $o = k$. $\square$

Proposition 3 (size of the large bundle). *Let (Y, U) be valid and $\omega = |J| - S$. Then $\omega = m - 2n + |NA|$ (with n, m the agents and goods of Stage L); every completion with owner o gives o at least $\omega + 2$ goods, and exactly $\omega + 2$ when the other terminals' slots are full. For $\omega \geq 1$, K3ALG's*

completion fills them when HITSET lists no good twice; otherwise slots can stay empty, and the owner gets one more good for each (Example 1: $\omega + 3$ goods, still EFX₀).

Proof. In a valid pre-allocation each good of NA is the pick of an agent outside U (not junk by (V1), not b_u, c_u by (V2)), which is then frozen, and each frozen agent's pick is in NA ; so $|F| = |NA|$. Let T_1 and T_0 be the terminals with and without a pick. Then $m = |U| + |F| + |T_1| + |U| + |J|$ and $S = |T_1| + 2|T_0|$, so $\omega = m - 2(|U| + |F| + |T_1| + |T_0|) + |F| = m - 2n + |NA|$. The other terminals take at most $S - \text{cap}(o)$ junk goods, exactly that many when their slots are full, so the owner gets its base and at least $|J| - (S - \text{cap}(o)) = \omega + \text{cap}(o)$ junk goods, with equality then; and $|\text{base}(o)| + \text{cap}(o) = 2$ for every terminal or upgraded agent. $\square$

The rotation does not increase NA (step (b) of the proof of Theorem 6), so it does not increase this bound (`EFX.LB.BadCase.omega_1e`). Lean proves the count and both bounds for every ω , and equality for K3ALG's completion when HITSET lists no good twice (`EFX.LB.complete_owner_length`); the next example is checked by `decide` in `EFX.K3.Examples.repeatedGood_algo`.

Example 1 (a repeated good). Four agents and nine goods: agent i values g_{2i+1}, g_0, g_{2i+2} at $4, 3, 2$. Nobody is peeled, and each agent in turn is inserted and picks g_{2i+1} ; $J = \{g_0, g_2, g_4, g_6, g_8\}$, $NA = \emptyset$, $S = 4$, $\omega = 1$. For $r = 3$, $E_3 = \{0, 1, 2\}$ with $\pi_x = \{g_0, g_{2x+2}\}$, so $\text{HITSET}(E_3) = (g_0, g_0)$: agent 0's slot gets g_0 , agent 1's stays empty, agent 2's gets g_2 , and the owner 3 gets g_7, g_4, g_6, g_8 , that is, $\omega + 3$ goods.

C State Dictionary and Two Runs

Object	Definition	Role
pick Y_i	i 's favourite good of R_i still unpicked at its turn in Phase 1, or $\perp$	i 's base good
block, leader	an insertion step with the R1 steps after it; its inserted agent	needs stay in a block (B); exposed agents are leaders
upgraded U	agents u with $Y_u = b_u$ that also get c_u (L3: $c_u \in J$, $b_u \notin NA$)	safe with $\{b_u, c_u\}$; no slots
junk J	goods unallocated after Phase 1 and the upgrades: no pick and no to the owner	must go into slots or
needs N_i	c_u $\{g \in R_i : g \succ_i Y_i\}$ (R_i if $Y_i = \perp$), for $i \notin U$	goods i lost; must sit alone
NA	$\bigcup_{i \notin U} N_i$, goods needed alone	never in a bundle of ≥ 2 goods
frozen	$i \notin U$ with $Y_i \in NA$	keeps its pick alone
terminal	$i \notin U$ with $Y_i \notin NA$ or $Y_i = \perp$	may take junk; ends need chains

slots $\text{cap}(i)$	1 (2) for a terminal with (without) a pick; 0 otherwise	junk a non-owner can take
S, ω	$\sum_i \text{cap}(i); \omega = J - S$	owner needed iff $\omega \geq 1$
base	$\{Y_i\} (\emptyset)$ for $i \notin U$; $\{b_i, c_i\}$ for $i \in U$	what i keeps in any completion
owner o	terminal or upgraded agent that takes the junk left after the other goods slots	only bundle of > 2
owner constraint	no $x \notin U, x \neq o, Y_x = a_x$, has $\{b_x, c_x\} \subseteq X_o$	excludes the only threat
exposed E_o	$x \notin U, x \neq o, Y_x = a_x, \{b_x, c_x\} \subseteq J \cup \text{base}(o)$	agents (OC) must protect
π_x	$\{b_x, c_x\} \cap J$	a good of it in a slot protects x
need chain	distinct $x_0, \dots, x_t; x_{s+1} \notin U$ needs $Y_{x_s}; x_0, \dots, x_{t-1}$ frozen, x_t terminal	gives slots (Thm. 5); rotation (Thm. 6)
valid	(V1) $J \cap NA = \emptyset$; (V2) $b_u, c_u \notin NA$ for $u \in U$	hypothesis of Thm. 4

The next two runs are instances from the repository whose outputs are machine-checked by `decide`, in `EFX.K3.Examples.peelOwner_algo` and `EFX.K3.Examples.rotOwner_algo`; we computed the intermediate states by hand from the definitions and cross-checked them with the repository's literal transcription of the Lean program. Agents are numbered from 0, as in Lean, and goods are $g_0, g_1, \dots$; unlisted values are 0.

Example 2 (a peel, then owner r). Six goods $g_0, \dots, g_5$. Agent 0 values g_2, g_4, g_5 at 84, 73, 54; agent 1 values only g_1 , at 87; agent 2 values g_4, g_2, g_3 at 92, 83, 44. *Stage R:* R1 fails for agent 0 ($84 < 73 + 54$) and applies to agent 1, which is peeled with $X_1 = \{g_1\}$; then R1 fails for agents 0 and 2 ($92 < 83 + 44$). *Phase 1:* 0 ranks $g_2 \succ g_4 \succ g_5$ and 2 ranks $g_4 \succ g_2 \succ g_3$. Both have all goods, so agent 0 is inserted and picks g_2 ; agent 2 then has two goods left and picks g_4 in an R1 step. One block, led by 0. No upgrade (no pick is a b). $J = \{g_0, g_3, g_5\}$, $NA = \emptyset$, both agents are terminals with one slot, $S = 2 < 3$. *Owner:* $r = 2$; $E_2 = \{0\}$, since $Y_0 = a_0$ and $\{b_0, c_0\} = \{g_4, g_5\} \subseteq J \cup \{Y_2\}$; $\pi_0 = \{g_5\}$ and $\text{HITSET} = (g_5)$, which fits $S - \text{cap}(2) = 1$. *Completion:* agent 0's slot gets g_5 and the owner gets g_0, g_3 : $X_0 = \{g_2, g_5\}$, $X_1 = \{g_1\}$, $X_2 = \{g_4, g_0, g_3\}$, with $3 = \omega + 2$ goods for the owner. It is EFX₀: agent 0 has 138 and $v_0(X_2) = 73$; agent 2 has

136 and $v_2(X_0) = 83$. Filling the slot with g_0 first instead would give $X_2 = \{g_4, g_3, g_5\}$ and $v_0(X_2 \setminus \{g_3\}) = 127 > 84 = v_0(\{g_2, g_0\})$.

Example 3 (a rotation with owner k^).* Six goods $g_0, \dots, g_5$. Agent 0 values g_1, g_2, g_0 at 5, 4, 2; agents 1 and 2 both value g_1, g_3, g_2 at 3, 3, 2 (so $a = g_1, b = g_3, c = g_2$, the tie broken by index). R1 fails for all ($5 < 6, 3 < 5$). *Phase 1:* agent 0 is inserted and picks g_1 ; agents 1 and 2 follow in R1 steps and pick $g_3 (= b_1)$ and $g_2 (= c_2)$. One block. No upgrade: $c_1 = g_2$ is picked. $J = \{g_0, g_4, g_5\}$, $N_1 = \{g_1\}$, $N_2 = \{g_1, g_3\}$, $NA = \{g_1, g_3\}$; agents 0 and 1 are frozen, agent 2 is a terminal with one slot, $S = 1 < 3$. *Owner test:* $r = 2$, $E_2 = \{0\}$ ($\{b_0, c_0\} = \{g_2, g_0\} \subseteq J \cup \{Y_2\}$), $\pi_0 = \{g_0\}$, $|E_2| = 1 > S - \text{cap}(2) = 0$, and no two sets meet, so r fails (Proposition 1); indeed $X_2 = \{g_2, g_0, g_4, g_5\}$ would give $v_0(X_2 \setminus \{g_4\}) = 6 > 5$. *Rotation:* $k^* = 0$; the chain is $0 \rightarrow 1 \rightarrow 2$ (agent 1 needs $g_1 = Y_0$, agent 2 needs $g_3 = Y_1$ and is a terminal). Agent 1 takes g_1 , agent 2 takes g_3 , and agent 0 takes $Y'_0 = g_2$ with upgrade good g_0 . Now $J' = \{g_4, g_5\}$, $NA' = \{g_1\}$, agent 1 is frozen, agent 2 is a terminal with one slot, $S' = 1 < 2$; $E'_0 = \emptyset$, so HITSET is empty. *Completion:* agent 2's slot gets g_4 and the owner 0 gets g_5 : $X_0 = \{g_0, g_2, g_5\}$, $X_1 = \{g_1\}$, $X_2 = \{g_3, g_4\}$. It is EFX₀: agent 1 has 3, $v_1(X_0) = 2$, and $v_1(X_2 \setminus \{g_4\}) = 3$; agent 2 has 3 and $v_2(X_0) = 2$; agent 0 has 6 and $v_0(X_2) = 0$.

D Running Time: Cost Table and Measurements

The program is the listing of Section 3, formalized as `EFX.K3.algoC`; see `proofs/k3_algorithm.md` of [10]. In L5 and L6 the program completes with owner r (or without owner) if k (or r) does not exist; by Lemma 4 and Theorem 5 this never happens.

Cost table (Lean bounds, with $N = n+m+1$ and finer). Stage R: $15N^2$ per round, $O(n^2m)$ in all. Rankings: $14N^2$. Processing order: $15N^3$. Phase 1's picks: $17N^3$. Upgrades: $36N^4$, from at most n rounds, each testing n candidates, each recomputing whether $b_k \in NA$ by scanning n agents with a membership test in U ; $n^4 + 17n^3 + n^2m + O(n^2 + nm)$ in the finer count (`EFX.K3.lbUpC_cost_fine`), the source of the n^4 term. Junk and slots: $O(N^3)$. Owner test: $20N^2 + 24N^3$. Blocks, k^* , chain, rotation, second owner test, completion: $O(N^3)$. Output: $4N^2$. The total is at most $400N^4$ (`EFX.K3.algoC_cost`), also at most $6400(n+m)^4$ (`EFX.K3.algoC_cost'`). Re-bounding every stage with a bound n on the lists of agents ($n+1$ for the upgraded agents after the rotation) and m on the lists of goods gives at most $n^4 + 20n^3 + 25n^2m + 124n^2 + 47nm + 119n + 22m + 3 \leq$

$270(n^4 + n^2m)$ operations (`EFX.K3.algoC_cost_fine`, `EFX.K3.algoC_cost_fine''`); the n^2m terms come from Stage R, the processing order, Phase 1's picks, the blocks, and the search for two meeting sets π_x in both owner tests, so the second owner test costs $O(n^2m + n^3)$. Measured on random instances, the count is far below the bound (ratio to N^4 : 0.002 at $n = 30$).

The accounting discipline. The cost monad `EFX.Timed` (`Timed.lean`) pairs a value with a count; `tick k` adds k , and sequencing adds the counts of the steps actually evaluated. The program keeps to four rules. (1) Every list traversal goes through a charged combinator (`mapC`, `filterC`, `anyC`, `findC`, `memC`, `eraseC`, `sumMapC`, `lengthC`, ...), one unit per cell visited plus the cost of the element function; the recursions of Phase 1, the blocks, the chain, the rotation and the completion charge one unit per step. (2) Every call of an uncounted function sits under a `tick` that bounds its work: `sort3C` is `tick 10` (at most four list cells, three value reads, three comparisons), `rankC` is `tick 6` (three table reads, three comparisons), `finRangeCk` is `tick k`. (3) No recursion over lists runs outside the monad; each counted definition has a value lemma saying that it computes the uncounted one, so every loop of the specification is a counted loop. (4) Tables (rankings, picks, blocks, slots, the output) are arrays filled once by `mkTable`, at the cost of their entries plus one per entry, and each read costs one. Abstracted, as disclosed in the files: arithmetic on natural numbers of any size (one unit), reading $v_i(g)$ (one unit), building pairs, options, list cells and closures, matching on them, and Boolean connectives. An auditor should read each `tick` and each `pure` in `Timed.lean`, `K3CostLB.lean` and `K3Cost.lean` and check that it covers the work it stands for; the value lemmas and the bound are then machine-checked.

Corollary 2 (bit complexity; written, not formalized). *With b the largest bit length of an input value, $K3ALG$ takes $O((n+m)^4(b+\log(n+m)))$ bit operations if each operation costs the bit length of its operands; for real values, the same holds with $b = 3$ for $K3ALG$ on the surrogate w of Corollary 3.*

Proof (of Corollary 2). We read the counted program (`K3Cost.lean`, `K3CostLB.lean`, `Timed.lean`); it forms no products. Values enter only in Stage R and in L1: each is compared with 0 or with another value of the same agent, and R1 compares $v_i(p)$ with $v_i(M \setminus \{p\})$, formed by adding values of agent i over the remaining goods. So every number derived from values is a sum of at most m values of one agent, below $m2^b$,

and Stage L reads no values. The other numbers are ranks in $\{0, \dots, 3\}$, slot counts and their sums (at most $2n$), list lengths (at most $n + m$) and block counters (at most n), and agents and goods are indices below n and m . So every operand, including table addresses, has $O(b + \log(n + m))$ bits, and each unit operation costs that many bit operations; multiply by Theorem 3's bound. $\square$

Corollary 3 (real values, comparison model). *Let every agent positively value at most three goods, with nonnegative values in $\mathbb{R}_{\geq 0}$ or in any linearly ordered cancellative commutative monoid. Let an oracle answer at unit cost whether $x \leq y$, where x and y are sums of at most two values of one agent (the same value twice allowed for agents with one or two relevant goods), and let adding two values of one agent cost one operation. Then a program computes, with at most $n(m + 12)$ oracle calls (the coefficient of the charge per call in `EFX.K3.surrogateC_cost`, which holds for every charge), natural-number valuations w that agree with v on every comparison of subset sums (as in Lemma 2), and `K3ALG` on w returns an allocation that is EFX_0 for v ; all in at most $n(m + 12) + 10nm + 971n + 400(n + m + 1)^4$ operations, oracle calls included.*

Proof (of Corollary 3). The program, `EFX.K3.algoOrdC`, inspects values only through the oracle and by additions. For each agent i it finds R_i with m calls $v_i(g) \leq 0$. For the values x_1, x_2, x_3 of i 's first three relevant goods (with one or two, the missing slots repeat the first good) it makes six additions and twelve calls: $x_j \leq x_k$ for $j \neq k$, and $x_j \leq x_k + x_l$ and $x_k + x_l \leq x_j$ for $\{j, k, l\} = \{1, 2, 3\}$. It sets w_i on these goods to the first of the 31 representatives of `EFX.Pat.reps` with the same answers (entries at most 5), and $w_i = 0$ elsewhere, stored as a table. This costs at most $n(m + 12) + 10nm + 971n$ operations (`EFX.K3.surrogateC_cost`), and w_i agrees with v_i on every comparison of subset sums (`EFX.K3.agree_surrogate`). `K3ALG` on w takes at most $400(n + m + 1)^4$ operations (Theorem 3), and its output is EFX_0 for w , hence for v (`EFX.efx0_iff_of_agree`); in Lean, `EFX.K3.algoOrd_efx0` and `EFX.K3.algoOrdC_cost`. The written argument needs only $m + 6$ comparisons of subset sums per agent (m to find R_i ; Appendix A); for agents with one or two relevant goods the program also asks comparisons such as $v_i(g_2) \leq 2v_i(g_1)$, so its oracle is of the real-RAM kind, not a pure subset-sum oracle. $\square$

Layer (iii): a faster implementation. A Python implementation of the same algorithm replaces the naive scans by heaps (for Stage R and the processing order), per-good counters of the agents that rank a good above

their pick (for the upgrades) and an index from junk goods to exposed agents (for HITSET). It takes each agent’s relevant goods with their values as input. Its written analysis, not formalized, gives $O((n + m) \log n)$ operations after reading the input; reading a dense $n \times m$ table would already cost nm . It agrees with a literal transcription of the Lean definitions on 200,000 random instances and, with Lean’s own evaluation of EFX.K3.algo, on 675 instances; every output passes a raw EFX₀ check, and every Lean operation count is below the Lean-checked bound. It takes 1.53s and 1.58s (medians) on random cores with 10^5 agents and 1.5×10^5 and 2×10^5 goods. These are measurements.

E The Lean Formalization

Trusted base. The file `lean/EFX/Model.lean`, copied from [11] with the namespace renamed, defines:

```
def finSum : (k : Nat) → (Fin k → Nat) → Nat
  | 0, _ => 0
  | k+1, f => finSum k (fun i => f i.castSucc)
              + f (Fin.last k)

structure Inst where
  n : Nat
  m : Nat
  v : Fin n → Fin m → Nat
abbrev Alloc := Fin I.m → Fin I.n
def bundleVal (X : I.Alloc) (i j : Fin I.n)
  (ex : Option (Fin I.m)) : Nat :=
  finSum I.m (fun g =>
    if X g = j ∧ ex ≠ some g then I.v i g else 0)
def EFX0 (X : I.Alloc) : Prop :=
  ∀ i j : Fin I.n, i ≠ j → ∀ g : Fin I.m, X g = j →
    I.bundleVal X i j (some g)
    ≤ I.bundleVal X i i none
def numRelevant (I : Inst) (i : Fin I.n) : Nat :=
  finSum I.m (fun g => if 0 < I.v i g then 1 else 0)
```

Main statements.

```
theorem EFX.target (I : Inst) (hn : 0 < I.n)
  (h : ∀ i, numRelevant I i ≤ 3) :
  ∃ X : I.Alloc, I.EFX0 X
```

```

theorem EFX.LB.corollaryD (I : Inst) (hn : 0 < I.n)
  (h3 : ∀ i, numRelevant I i = 3)
  (hbal : ∀ i g, 2 * I.v i g ≤ finSum I.m (I.v i)) :
  ∃ X : I.Alloc, I.EFX0 X ∧ ∃ o, ∀ j, j ≠ o →
    finSum I.m (fun g => if X g = j then 1 else 0)
      ≤ 2
def EFX.K3.algo (I : Inst) (hn : 0 < I.n) : I.Alloc :=
  (algoC I hn).val
theorem EFX.K3.algo_efx0 (I : Inst) (hn : 0 < I.n)
  (h : ∀ i, numRelevant I i ≤ 3) :
  I.EFX0 (algo I hn)
theorem EFX.K3.algoC_cost (I : Inst) (hn : 0 < I.n) :
  (algoC I hn).cost ≤ 400 * (I.n + I.m + 1) ^ 4
theorem EFX.K3.algoC_cost_fine'' (I : Inst) (hn : 0 < I.n) :
  (algoC I hn).cost ≤ 270 * (I.n ^ 4 + I.n ^ 2 * I.m)

```

`EFX.target_ordered` and `EFX.corollaryD_ordered` state the same over any type with 0 , $+$ and $\leq$ satisfying the axioms of a linearly ordered cancellative commutative monoid (class `EFX.OrderedValue`: $+$ associative and commutative with identity 0 , $\leq$ a total order, and $a \leq b \iff a + c \leq b + c$), with nonnegativity as a hypothesis, relevance as $v_i(g) > 0$ and balance as $v_i(g) + v_i(g) \leq v_i(M)$; at $\mathbb{N}$ they are exactly the statements above. $\mathbb{R}_{\geq 0}$ with its addition and order satisfies these axioms; core Lean has no real numbers, so this last step is not a Lean theorem.

Theorems cited (all in `lean/EFX/`, file in parentheses). Every numbered result of this paper is a Lean-checked theorem, for natural-number values unless Table 1 or the list below says otherwise, except those whose heading or row in Table 1 gives another label. Every declaration of the library depends only on `propext`, `Classical.choice` and `Quot.sound`, which the repository’s check script verifies over all declarations; the main results also carry `#print axioms` certificates.¹

¹ The repository’s `LEDGER.md` lists, for each result, its written proof, its Lean names and its checks.

Result	Lean names (file in <code>lean/EFX/</code>)
Thm. 1	<code>EFX.target (Target); EFX.target_ordered (RealValues)</code>
Thm. 2	<code>EFX.LB.corollaryD (CorollaryD); EFX.corollaryD_ordered (RealValues)</code>
Thm. 3, correctness	<code>EFX.K3.algo_efx0, EFX.K3.algo_eq_spec (K3Cost)</code>
Thm. 3, operations	<code>EFX.K3.algoC_cost (K3CostBound)</code>
finer count (Sect. 5)	<code>EFX.K3.algoC_cost_fine, EFX.K3.algoC_cost_fine'', EFX.K3.lbUpC_cost_fine (K3CostFine)</code>
Cor. 1	<code>EFX.K3.algoRat_efx0, EFX.ratScale_agree, EFX.ratScale_efx0_iff (K3Extras)</code>
Cor. 3	<code>EFX.K3.algoOrd_efx0, EFX.K3.algoOrdC_cost, EFX.K3.surrogateC_cost, EFX.K3.agree_surrogate (K3Real); EFX.K3.algoOrdC_cost_fine (K3CostFine)</code>
$k \leq 2$ (Sect. 1), every order and choice of favourite	<code>EFX.Inst.sdRun_efx0, EFX.Inst.serialDict_efx0 (K3Extras); EFX.exists_efx0_of_count (Bridge)</code>
Lem. 1	<code>EFX.peelBundle, EFX.peelEmpty, EFX.peelR2 (PeelingR2); EFX.peel (Peeling)</code>
Lem. 5	<code>EFX.junk, EFX.Inst.exists_efx0_of_junk (Junk)</code>
Prop. 2	<code>EFX.core_reduction (Target)</code>
Lem. 2, any OrderedValue	<code>EFX.l12, EFX.efx0_iff_of_agree (RealValues)</code>

Lem. 3	EFX.LB.complete_some (PreAlloc)
Lem. 6	EFX.LB.upFinal_valid (Blocks)
Lem. 4	EFX.LB.lastOut_exists, EFX.LB.lastOut_terminal (OwnerR)
Thm. 4	EFX.LB.Valid.sound (PreAlloc)
Thm. 5	EFX.LB.theoremA_invalid, EFX.LB.exposed_lead, EFX.LB.exposed_r, EFX.LB.exposed_blk_inj, EFX.LB.isChain_props, EFX.LB.outside_terminals, EFX.LB.exposed_split (OwnerR)
Thm. 6	EFX.LB.theoremB (Rotation)
Thm. 7	EFX.LB.lbPlusRun_sound, EFX.LB.lbPlusOut_exists (LBPlus)
Prop. 1, HITSET	EFX.LB.validOwner_iff, EFX.LB.hitSet_fits, EFX.LB.hitSet_length, EFX.LB.length_le_of_hits (OwnerR)
Prop. 3	EFX.LB.largeBundle_size, EFX.LB.omega_eq, EFX.LB.Completion.owner_length_ge, EFX.LB.Completion.owner_length_eq, EFX.LB.complete_owner_length, EFX.LB.BadCase.omega_le, EFX.K3.Examples.repeatedGood_algo (K3Extras)
Examples 2, 3	EFX.K3.Examples.peelOwner_algo, EFX.K3.Examples.rotOwner_algo (K3Examples)
Independent statement	Audit.target_audit, Audit.corollaryD_audit (Audit)
Second proof	EFX.C4min.corollaryD_K3, EFX.C4min.target_K3 (K3Theorem)

EFX.K3.algoRat_efx0 is for core Lean's rationals (OInst Rat), with values scaled by the product of their denominators; EFX.K3.algoOrd_efx0 assumes a correct oracle, $\text{le } x \ y = \text{true}$ iff $x \leq y$. EFX.LB.lbPlusRun_sound quantifies over all runs of LB^+ (any Phase 1 order with R1 priority, any upgrade order, any need chain, any completion satisfying (OC)); EFX.LB.lbPlusOut_exists shows that an output always exists. The independent statement writes bundles as lists that partition the goods and EFX_0 with List.erase, and is derived from EFX.target and EFX.LB.corollaryD. The second proof establishes Theorem 1 again, and Theo-

rem 2 for cores, through Pareto-maximal pre-allocations, independently of LB^+ .

Not formalized. Corollary 2; the analysis and the speed of the Python implementation; that K3ALG, run directly on rational or real values, would make the same decisions as on the scaled values or on w ; the cost of computing the scaling; that $\mathbb{R}_{\geq 0}$ satisfies the axioms of `EFX.OrderedValue`; serial dictatorship for $k \leq 2$ with values other than natural numbers; Lemma 7 and Proposition 4; for the certification, that the allocations of a disconnected core’s components combine and that a core with $m = 2n$ has an allocation giving every agent two of its goods; and that each `tick` of the counted program covers its work.

Kernel checks. The repository’s check script also replays the whole library in the replay checker `leanchecker --fresh` of Lean. At commit `bf81efb` of [10], the kernel `nanoda 0.4.19` (written in Rust, independent of Lean’s C++ kernel) re-checked, from the export of `lean4export`, all 2,934 named declarations of the library, which include everything formalized in this paper (Theorems 1–3, K3ALG’s cost bounds, the ordered-value transfer, the corollaries and the independent statement), together with their dependencies: 8,762 declarations, with no error; the log is `results/audit_kernel_2026-09-27.log`. As negative controls, `nanoda` rejected a `native_decide` proof and a proof of $1 = 2$ that bypasses the elaborator’s kernel check; `leanchecker` rejected the second, and the axiom check catches the first. The files formalizing Theorems 1–3, their proofs and the independent statement have about 9,600 lines.

F Computational Certification

Each row below is verified by exhaustive computation: certificates are stored allocations, re-checked from the raw EFX₀ definition without a SAT solver, and the enumeration of connected cores is checked for completeness by a second, independent method. By Lemma 7, EFX₀ in a core depends only on the strict ranking profile, and its cases stay sufficient when values tie (`proofs/lemmas.md`, L5, of [10]), so certificates for strict profiles cover tied values too; a disconnected core is covered by its components.

Claim and scope	Size
EFX ₀ exists, every 3-relevant in-stance, $n \leq 6$	251 + 3,182 connected cores, and 3 with $m = 2n$ (below)
EFX ₀ exists, every 3-relevant in-stance, $n \leq 7$	adds $n = 7$, by the rows on $n = 7$ below
Theorem 2, every core with $n \leq 6$, connected or not	certificates, components combined
Theorem 2, connected cores, $n = 7$, $11 \leq m \leq 13$	3,103, 541 and 37 cores ($m = 11, 12, 13$); 6^7 profiles each
Theorem 2, connected cores, $n = 8$, $m \in \{14, 15\}$	1,232 and 52 cores
LB never fails, cores with $n \leq 6$	3,436 connected, 131 disconnected
LB never fails, connected cores with $n = 7$	41,170 cores, 1.15×10^{10} pairs
LB never fails, connected cores, $n = 8$, $m \geq 13$	12,763 cores, 2.14×10^{10} pairs

LB is the repository’s earlier construction (`proofs/construction.md §3` of [10]): Phase 1 with a lookahead insertion rule, the same upgrades, and a search over owners instead of the rotation. For the rows on LB, the certificates (LB’s outputs), re-checked independently, confirm Theorem 2 in every labelling; that LB itself never fails is counted by the construction program, which checks each output against the raw definition, and holds for the labelling stored in the certificates, as LB breaks ties by index. Cores with $m = 2n$ need no computation: counting shows that each agent then has one private good and values two shared goods, each valued by exactly two agents, so (a 2-regular bipartite graph has a perfect matching) every agent can receive its private good and one shared good, and an agent holding two of its three relevant goods is safe (`EFX.safe_of_two_own`).

Tests (measurement). 5,844,263,904 runs of a literal implementation of LB⁺ (every core with $n \leq 6$ under every ranking profile, with every sequence of insertion choices and with LB’s own Phase 1 order; random choices on the cores with $n \in \{5, 6\}$; LB’s order and random choices on the connected cores with $n = 7$, $m \geq 11$, and with $n = 8$, $m \in \{15, 16\}$; random non-core instances), asserting every lemma of Section 4 and checking every output against the raw definition: 1,327,101 rotations, 0 failures. An independent Python implementation gives identical tallies

for $n \leq 5$. K3ALG itself returned an EFX₀ allocation on all 152,071,632 ranking profiles of all cores with $n \leq 6$.

Counterexamples (each with a checker in [10]):

- bundles of at most two goods suffice in a core: false for $n = 3$, $m = 5$ (Proposition 4(a); 14 of 216 profiles, found by SAT and by a SAT-free brute force); no core with $n = 2$ fails;
- bundles of at most two goods suffice when $m \leq 2n - 2$: false, smallest $n = 4$, $m = 6$; at $n = 6$, $m = 10$, 57 of the 211 connected cores have a failing profile (confirmed by a second encoding);
- the same for instances that are not cores: four identical agents with values 4, 3, 2 on three goods, and three goods valued by nobody;
- one bundle of three goods suffices: false for $n = 4$, $m = 7$ (Proposition 4(b)); at $n = 6$, $m = 11$, 3 of 25 cores need four (confirmed by a second encoding that enumerates every allocation with one large bundle).

G Ordinality of Cores and the Counterexamples

A good is *alone* if it is the only good of its bundle.

Lemma 7 (cases of safety; written proof, not formalized). *Let agent i have $R_i = \{a, b, c\}$ with $v_i(a) > v_i(b) > v_i(c) > 0$ and $v_i(a) < v_i(b) + v_i(c)$. Then i satisfies the EFX₀ constraints iff one of: (T) i holds a , and either i holds b or c , or b and c are not together in a bundle of at least three goods; (P) i holds b and c ; (B) i holds b , and a is alone; (C) i holds c , and a and b are alone; (E) a , b and c are all alone.*

Proof. The subset sums are ordered $0 < c < b < a < b + c < a + c < a + b < a + b + c$ (writing x for $v_i(x)$), so every constraint is decided by the ranking. Consider the goods of R_i that i holds. If it holds a and one more, every other bundle meets R_i in one good, worth $\leq b < a + c$. If it holds $\{b, c\}$, the only threat is $a < b + c$. If it holds only a : bundles meeting R_i in one good are harmless; $\{b, c\}$ as a bundle threatens $b < a$; b, c together with another good threaten $b + c > a$. If it holds only b : a not alone lies with c (threat a) or with a good outside R_i (threat $\geq a$), both $> b$; if a is alone, c 's bundle threatens at most $c < b$. If it holds only c , it needs a and b alone by the same argument. If it holds none of a, b, c , any bundle with a good of R_i and at least two goods threatens a positive amount, so all three must be alone. $\square$

The cases remain sufficient with ties, so tied cores reduce to strict ranking profiles (`proofs/lemmas.md`, L5).

Proposition 4 (limits of the shape; written proof, not formalized). *Take strict rankings and strictly balanced values. (a) Let agents $0, 1, 2$ rank $g_0 \succ g_1 \succ p_i$, where p_i is private to agent i . No EFX_0 allocation has all bundles of at most two goods, while $\{g_0\}, \{g_1\}, \{p_0, p_1, p_2\}$ is EFX_0 . (b) Let agents $0, 1, 2, 3$ rank $g_0 \succ g_2 \succ p_0$, $g_0 \succ g_2 \succ p_1$, $g_1 \succ g_2 \succ p_2$, $g_1 \succ g_2 \succ p_3$, with p_i private to agent i . Every EFX_0 allocation with at most one bundle of more than two goods has bundle sizes $4, 1, 1, 1$.*

Exhaustive computation shows that no core with two agents needs a bundle of more than two goods, so (a) is a smallest example; see `proofs/counterexamples.md` and Section 1 of `proofs/construction.md` in [10].

Proof (of Proposition 4). (a) With five goods in three bundles of at most two goods, the sizes are $2, 2, 1$, so exactly one good is alone. Good g_0 is everyone's top and has one holder, so two agents do not hold their top. For them (T) fails, (C) and (E) need two or three alone goods, so each is in case (P) or (B), and both hold g_1 : impossible. In $\{g_0\}, \{g_1\}, \{p_0, p_1, p_2\}$, the holder of g_0 is in case (T), the holder of g_1 in case (B), and the third agent in case (C). (b) Each of the tops g_0, g_1 has a claimant that does not hold it (a *loser*). A loser is safe only by (P) or (B), which need it to hold g_2 , or by (C) or (E), which need g_2 alone. The two losers are distinct and cannot both hold g_2 , so g_2 is alone, and no agent is in case (P). A loser holding $\{g_2\}$ is in case (B) or (E) and needs its top alone; a loser not holding g_2 is in case (C) or (E) and needs its top and g_2 alone. So g_0, g_1, g_2 are alone, held by three agents, and the fourth bundle is $\{p_0, p_1, p_2, p_3\}$. The allocation giving g_0 to agent 0, g_1 to agent 2, g_2 to agent 1 and the private goods to agent 3 is EFX_0 (cases T, B, T, C). $\square$

H Related Results at a Glance

Where the difficulty lies. Under ordinary EFX, a good that agent i values at zero never constrains i , so goods valued by some agents and not others are harmless; this is why [15] can give the goods nobody values to an arbitrary agent, and on multigraphs ordinary EFX is easy, so the difficulty there is EFX_0 [14]. Under EFX_0 with complete allocations, every such good must go into some bundle, and it turns the constraint of each agent that does not value it into envy-freeness towards that bundle. The

Table 2. Existence results for complete allocations under restrictions. *Version cited:* the published version where one exists, else arXiv. ^aGoods nobody values go to an arbitrary agent. ^bIncludes additive. ^cExtended abstract with a proof sketch. ^dAn AAMAS 2025 extended abstract has the degree and girth conditions. ^eAccepted at EC 2026.

Result	Restriction (agents, goods, who values what)	Notion	Complete	Valuations	Algorithm	Version cited
[5]	$n = 3$	EFX ₀	yes	additive	pseudo-polynomial	JACM 2024
[7]	at most three distinct valuations	EFX ₀	yes	additive	no bound stated	EC 2025
[13]	$m \leq n + 3$	EFX ₀	yes	monotone	none stated	MOR 2024
[3]	$n = 4, m \leq 9$	EFX ₀	yes	additive	none stated	arXiv
[6]	graphs: at most two valuers per good, two agents share at most one good	EFX ₀	yes	monotone	polynomial time	MOR 2026 (EC 2023)
[1]	multigraphs: at most two valuers per good	EFX ₀	yes	cancelable ^b	polynomial time	arXiv ^e
[14]	multigraphs: bipartite, or at most $\lceil n/4 \rceil - 1$ neighbours per agent, or girth ≥ 6 (non-parallel edges)	EFX ₀	yes	monotone	not efficient (stated)	arXiv ^d
[8]	at most two valuers per good, $v_i(g) \in \{0, v_g\}$	EFX ₀	yes	restricted additive	no time bound stated	WINE 2024
[9]	hypergraphs of girth at least four (a good joins its valuers)	EFX ₀	yes	monotone	polynomial time	AAAI 2026
[15]	at most four relevant goods per agent	EFX	yes ^a	additive	polynomial time, announced	AAMAS 2024 ^c
[11]	at most two relevant goods per agent	EFX ₀	yes	additive; monotone	serial dictatorship; Lean-checked	GitHub, v1.2.0
this paper	at most three relevant goods per agent; any number of valuers per good	EFX ₀	yes	additive	polynomial time; Lean-checked	this paper

structural results cope with this when each good has at most two valuers or the valuers form a hypergraph of girth at least four; the others bound n or m . Theorem 1 allows any number of valuers per good.